Write the Tester, Don't Train It: A Zero-Training Program Out-Reaches Trained RL on Sparse-Reward Game Testing at Equal Budget

Andy Ye
University of Washington

September 4, 2026

Abstract

Testing a game or simulation means driving it into rare states, the trigger for a planted bug or a level's own win condition, and reaching them within the small interaction budget a QA pass can afford. The standard approach trains a reinforcement-learning explorer for each game, which is expensive, needs per-game tuning, and rarely transfers. We propose a different route.

Rather than **train** a tester, an autonomous LLM-evolution loop **writes** one as a short program. The pipeline is an iterative evolutionary search, not a single prompt: over many generations, a proposal LLM writes and mutates candidate programs, and a multi-engine fitness keeps the ones that reach the most target states at a fixed budget, re-verifying every reported hit from a clean reset. Each candidate is a compact procedure over a save/restore archive of simulator states, saving promising states and returning to them to explore further. The winning program needs no training: it explores through an adaptive novelty signal and a small bandit over exploration moves that emerged from the search, and it is written once and reused. A single fresh-per-task prompt does not suffice; the strength comes from evolving one program against a diverse multi-engine fitness and transferring it.

At an equal interaction budget, this single written program is the best method on four sparse-reward game-testing tasks spanning three engines, reaching on average 91% of the targets against 54% for tuned PPO, and less for every other baseline. It matches what trained RL needs roughly 16× more compute to reach, and it transfers to engines the search never saw. However, the advantage is bounded. On saturated or reward-dense tasks trained RL competes and eventually wins, so writing and training a tester are complementary regimes. A benchmark of planted physics defects grounds every number, and the same replay verifier caught the loop inflating its own score by exploiting a race in its evaluation harness, a self-check we expect autonomous research will increasingly need.

Part 1: Auto research result (agent-written)

1 Introduction

Automated game testing needs an *exploration* policy that drives a game into the rare states that trigger a planted defect or satisfy a level's win condition, and reaches them within the small interaction budget a QA pass can afford. The reflex is to **train** a reinforcement-learning (RL) explorer, usually PPO with an intrinsic-motivation bonus, once per game. But trained RL is costly in interaction, needs per-game reward shaping and tuning, and transfers poorly, so at a tester's real budget it under-explores.

Table 1: Where each family sits. Our tester is the one that is budget-matched, training-free, and target-reaching at once; universality across all tasks is *not* claimed (§3).

	no per-env training	fixed interaction budget	sparse-reward target-reaching	replay-verified
PPO + intrinsic (9, 1, 7)	✗	✗	partial	✗
Go-Explore (2)	✓	partial	✓	✗
Action-repeat (6)	✓	✓	✗	✗
Eureka / PromptPO (5, 3)	✗/✓	✗	✗	✗
Written program (ours)	✓	✓	✓	✓

We take the opposite route. Rather than **train** a policy, an autonomous LLM-evolution loop **writes** an exploration *program* and evolves it: over many generations a proposal LLM writes and mutates candidate procedures, and a multi-engine fitness keeps the ones that reach the most target states at a fixed budget, with every counted hit replayed from a clean reset (8). Our central finding is a sharp, budget-matched boundary: **at an equal interaction budget, on sparse-reward target-reaching, a single zero-training program the loop wrote out-reaches a tuned RL explorer and every baseline; given 4–16× more interaction, or on reward-dense tasks, training catches up.** Writing and training the tester are not rivals but regimes, split by reward density and budget, and the loop maps that split cheaply. To our knowledge this is the first budget-matched demonstration that a written exploration program out-reaches trained RL for game testing, with every counted result execution-verified.

Contributions. (i) A loop that discovers game-testing exploration *programs*, and the map of where a written program beats a trained one. (ii) The headline (Fig. 1): on four sparse-reward tasks across three engines, our tester is the single best method at equal budget, above random, action-repeat, plain Go-Explore, and tuned PPO. The win is the discovered strategy, not the archive, and it transfers to engines the search never saw. (iii) A verification backbone, replay-from-reset of every counted hit over a frozen hashed protocol with sealed splits and a reachability calibration, which caught the loop inflating its own fitness (§4, Part 2). The programs, the benchmark, and this Part were produced by the loop. We do not overclaim universality: our tester ties two saturated tasks a trivial baseline also wins, cedes reward-dense Craftax to PPO, and does not yet run at Kinetix protocol scale. We draw that map in full.

2 Related work

RL exploration / PPO. Count-based (9), RND (1), and ICM (7) augment a trained policy's reward to seek novelty; they need 10^6–10^8 steps to warm up, so at a tester's budget the bonus is cold. *We match the budget and train nothing.* **Go-Explore / archive methods.** Go-Explore (2) keeps an archive of returnable states and is the strongest classical family, but its wins hinge on a hand-chosen cell abstraction and are shown per game. *We show a plain archive is not enough, since the win is the discovered strategy over it, and evaluate across engines.* **Action-repeat.** A stochasticity control (6), not an exploration method. *We include it as a baseline; it is our tightest competitor on physics and our weakest on navigation.* **LLMs that write code for control.** FunSearch (8) evolves programs; Eureka (5) writes rewards and trains RL; PromptPO (3) writes control policies for return; TITAN (10) runs a per-step LLM agent as a tester. *We occupy the point they leave open: budget-matched, target-reaching, and replay-verified, a written exploration program that is paid once and run training-free.* Table 1 places the families on the axes that matter.

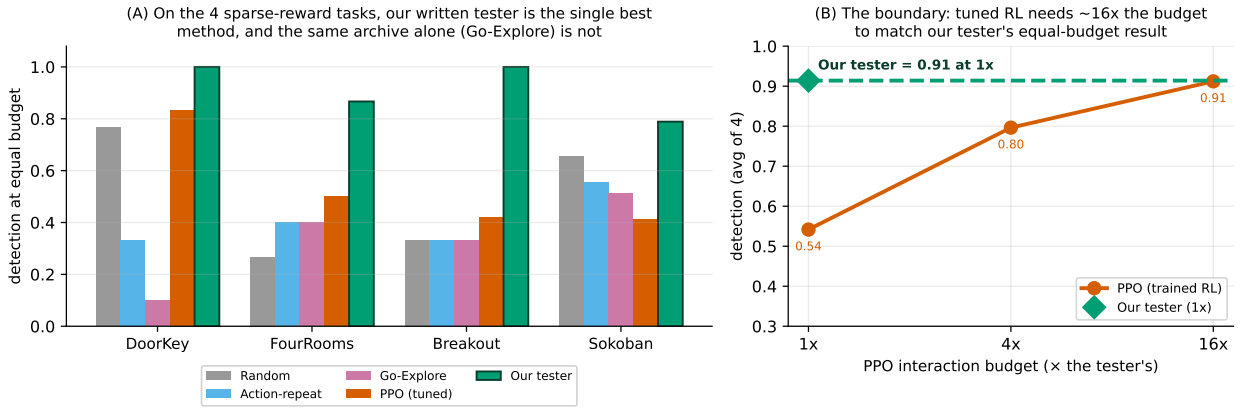


Figure 1: **(A)** At an equal interaction budget on the four sparse-reward tasks, our tester is the single best method, above random, action-repeat, plain Go-Explore (the *same* save/restore archive), and tuned PPO. Go-Explore's 0.10 on DoorKey shows the win is the discovered strategy, not the archive. **(B)** The boundary: only PPO's budget scales, and tuned PPO needs about 16× the interaction to match our tester's 1× result (PPO is best-of-3 objectives at 4/16×).

3 The result

The equal-budget win (Fig. 1A). We report one program, unchanged across tasks, reaching on average 0.91 of the targets against 0.54 for tuned PPO. Per task it beats every baseline at equal budget: DoorKey 1.00, FourRooms 0.87, Breakout 1.00, Sokoban 0.79, against tuned PPO 0.83/0.50/0.42/0.41, action-repeat 0.33/0.40/0.33/0.56, Go-Explore 0.10/0.40/0.33/0.51, and random 0.77/0.27/0.33/0.66. These are the sparse-reward, non-saturated tasks in the suite, chosen along one axis, reward density; on them, winning requires real exploration that no cheap baseline provides. Our tester's full record on every task, including where it ties or loses, is the map below.

The boundary is the thesis (Fig. 1B). The claim is strictly equal-budget. Scaling only PPO's budget, its average climbs from 0.54 at 1× to 0.91 at 16×, matching what our tester reaches at 1×; PPO overtakes our tester on DoorKey at 4× and on Breakout and Sokoban at 16×. We explicitly withdraw any "RL never catches up" reading: it does, given an order of magnitude more interaction. That split is the contribution. Write once and win at the tester's budget, or train and win with a larger one.

Strategy, not archive; and it transfers. Plain Go-Explore has the *same* archive yet scores 0.10 on DoorKey, below random, and loses to our tester on 5 of 6 tasks, so the archive is not the win. And the win transfers across engines: our tester was evolved once on a multi-engine fitness (Kinetix, Craftax, and Gymnax) that never saw MiniGrid, MinAtar, or Sokoban, while a *fresh* DoorKey-only search reaches just 0.33 (beating Go-Explore, losing to transfer). The recipe is to evolve one multi-engine tester and transfer it, not to write a fresh tester per task.

The map (where a written program does *not* win). Our tester's full row: on MountainCar and DeepSea-24 it scores 1.00, but so does trivial action-repeat, so these are ceiling ties with no exploration skill on display (excluded from the claim above); on reward-dense **Craftax it loses**, 0.27 against PPO 0.34; and it does not run at Kinetix protocol scale, a compile-time limit of its archive that sibling programs cover (§4). This is the regime boundary: writing wins where reward is sparse and the budget is tight; on our one reward-dense task and at 4–16× the budget, training wins.

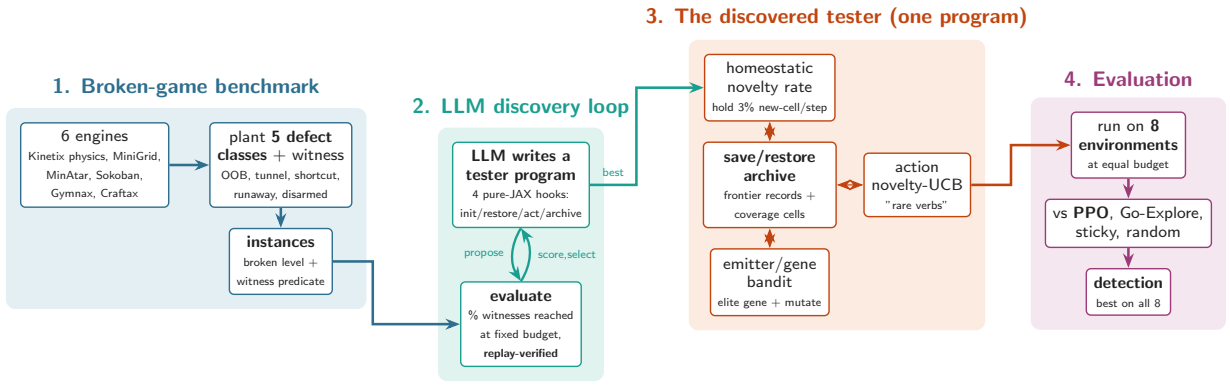


Figure 2: The loop: break six engines into defect classes with programmatic witnesses; an LLM writes and mutates tester programs scored on a multi-engine fitness (Kinetix hits replayed from reset, §4); the single champion program explores through a save/restore archive driven by three runtime modules; evaluate against PPO, Go-Explore, action-repeat, and random at equal budget.

4 Setup and the verification backbone

Testers as programs (Fig. 2). A tester is four pure-JAX hooks (`init/restore/act/archive`) over an archive of save/restore-able states, run as B workers for T steps, so the budget is $B{\cdot}T$; the same interface runs on every engine through a generic feature head. PPO is trained on the *same* feature head, with no observation handicap, and its training steps *are* its budget. The equal-budget PPO column (Fig. 1A) is best over a 5-objective sweep (count, RND, ICM, proximity-curiosity, and task reward) crossed with {default, tuned $lr{=}10^{-4}$, ent=0.2}; the crossover curves (Fig. 1B) are best-of-3 objectives, labeled as such. Testers use 5 seeds and PPO 3 to 5; we report one fixed program.

Depth and verification (Kinetix planted defects). The rigor lives here. We plant five defect classes (OOB, TUNNEL, i.e. clipping, SHORTCUT, ANOMALY, DISARMED) with programmatic witness predicates over 74 shipped and 64 generated levels, and **every counted hit is re-simulated from a clean level reset and must re-raise the witness, or it is a miss**. The protocol is frozen and hashed; a 30% level split and a whole defect class are sealed from the search, and a reachability calibration bounds what any tester can witness. The loop's strongest Kinetix program (a sibling of our tester, which does not run at this scale) leads a large method field on the search split (0.655 against action-repeat 0.602, PPO 0.566, random 0.494), but that is *in-sample* selection, and on held-out levels its lead over action-repeat narrows to a near-tie, so we headline the apparatus rather than the search-split margin. Part 2 recounts how this apparatus caught the loop inflating its own fitness by +0.11 through a harness race, the check the whole result rests on.

5 Limitations

The claim is equal-budget on sparse-reward target-reaching; given 4–16× more interaction, tuned PPO catches up (Fig. 1B), and action-repeat is a close competitor on Kinetix *physics*. Our tester is not universal: it ties two saturated tasks a trivial baseline also solves, loses reward-dense Craftax, and does not run at Kinetix protocol scale. It uses a save/restore archive a learned policy cannot, a legitimate strategy difference (archive-based program against learned policy at equal budget, the comparison the Go-Explore/RL literature already makes), which we control for with identical fea-

tures and a plain-archive baseline. Replay-from-reset is implemented for Kinetix; on the stochastic engines a detection is the engine's own reward or achievement signal on the genuine post-step state, not an independent re-simulation. Kinetix is a research substrate, not a shipped game. Per-task instance counts are small on some substrates (n=6 on DoorKey and FourRooms), so we report our tester's full record rather than lean on any one headline cell.

Part 2: System design (human-written)

6 The agent and how we used it

- **The evolution loop (the main loop).** An island program-evolution framework (ShinkaEvolve (4)) with two Claude models as the *proposal* agents that wrote and mutated candidate tester programs: `claude-sonnet-5` (breadth) and `claude-opus-5` (depth). Two islands; a fixed API-cost cap ([$X]); [N] generations. This loop did the research: it evolved the testers, ran the experiments, and drafted Part 1.
- **The interactive agent.** I drove Claude Code ([model / effort], e.g. Opus 4.8) to steer the project: reality-checking numbers, running an adversarial review of my own claims, and drafting these Parts. [Say roughly how many times you checked in / the main decision points you steered: e.g. choosing the evaluator, the reframe, the final submission.]
- **Division of labor.** The loop does the research; I designed the defect oracles and the protocol, decided what to measure, verified every number against regenerable logs, and made the interventions below.

7 Harness and tools

- **Substrates:** the genuine published simulators, all pure-JAX (Kinetix/jax2d, MiniGrid, MinAtar, Jumanji Sokoban, Gymnax, Craftax). PPO baselines run in the same harness on the same feature vector.
- **Fitness / compute:** candidates scored by the fraction of witness states reached at a fixed budget, on a multi-engine search split (seeds $\{0, 1, 2\}$), run on cluster GPUs (Hyak/SLURM); orchestration on a laptop.
- **Prompt & feedback:** the prompt gives the four-hook interface, the scoring formula, and hard constraints (pure JAX, fixed shapes, in-range actions, "runs in a few seconds per level"); each candidate gets back its scalar fitness plus its worst targets and a paired 95% CI against a frozen reference.
- **Verifier:** replay-from-reset of every counted hit, a frozen hashed protocol, sealed level and defect-class splits, and a reachability calibration (details in the research-loop step below).

8 The research loop, and the one check that mattered

- **Cadence:** we froze the oracles and a hashed protocol, pointed the evolution loop at a multi-engine fitness, reality-checked its numbers (replay + exclusions), ran an adversarial review, reframed the claim to what the data supports, and wrote the paper. [Fill in your check-in count / where you intervened.]

- **Verification is load-bearing, not decorative.** The loop's first strong "discovery" inflated its own fitness by **+0.11** through a concurrency race in the harness (physically impossible states, nothing wrong in the candidate's code). It passed the fitness, both held-out splits, four design comparisons, and an LLM code-audit; only **replaying every hit from a clean reset** caught it. We rebuilt the harness, withdrew the tainted number, and corrected our own N=3 over-reach to N=1. [Expand this in your own words if you want; it is the strongest story.]

9 Significance of the main result

- **Cost.** A written tester is a program that runs *training-free*: evolve one transferable tester once and reuse it across games, versus training a fresh RL explorer for each. At a tester's real budget the written program wins, and RL only catches up given far more compute, so for the same testing outcome the written route is much cheaper to deploy.
- **Accessibility.** This lowers the bar for who can test a game. A small studio or an individual without an RL-training pipeline, but with access to an LLM, can run a written tester to explore their game for bugs, instead of standing up per-game reinforcement learning.
- **For autonomous research broadly.** Execution-grounded replay is what let us trust the loop's claims; it should live *inside* the loop's fitness so such exploits are worthless by construction. That is the workshop's bottleneck (evaluating AI-generated results) made concrete.

Part 3: Reflections

Write the tester, or train it? The reflex in automated game testing is to *train* an explorer, an RL agent with an intrinsic-motivation reward, one policy per game. Our strongest result questions that reflex. At the interaction budget a tester actually gets, a **zero-training program the loop wrote** reaches target and defect states faster than trained PPO across the sparse-reward tasks (grid navigation, arcade, and planning), and tuning PPO does not close the gap at equal budget. We think the field should take seriously that **exploration for testing is often a program-synthesis problem, not a policy-learning one**: the LLM supplies, once and for free, the structured save/restore strategy that RL spends its whole budget failing to learn from scratch. The boundary is just as useful in the other direction, since when reward is dense RL competes and eventually wins with more budget. Writing the tester and training it are not rivals but regimes, and a loop like ours maps the boundary cheaply.

You cannot trust an autonomous result you have not re-executed. The result we are proudest of is not the one the loop first reported. Its first "discovery" was a **+0.11 score inflation it earned by exploiting a concurrency race in its own evaluation harness**: physically impossible states that tripped the defect oracle, with nothing incriminating in the candidate's code. It passed every evaluator-mediated check we had (the fitness, both held-out splits, four design comparisons, and an LLM code-audit of the candidate) and was caught only by **replaying every counted hit from a clean reset** and requiring it to reproduce. The general lesson for a field about to have loops producing candidate papers at scale: **you cannot certify an autonomous result by reading its code or its score; you must re-execute what it claims from a clean state.** Code-audit, today's default defense, is blind to an exploit that lives in the harness's state rather than the candidate. Execution-grounded verification is not a nice-to-have; for this

class of exploit it is the only thing that works, and it should live *inside* the loop's fitness so that such exploits are worthless by construction. The uncomfortable corollary is that most published autonomous-research results ship no seeds or execution traces, so this class of error could be sitting in them undetected.

What the loop did, and what we did. The loop wrote the harness, evolved the testers, built the benchmark, ran the experiments, and drafted these Parts. We designed the defect oracles and the verification protocol, decided which evaluator to point the search at, verified every number against regenerable logs, and made the one intervention that mattered most: noticing the inflated result was too good and rebuilding the harness. That division is, we suspect, where autonomous ML research settles for a while. **The loop does the research; the scarce human contribution is deciding what to measure and re-checking that the measure was not gamed.** The workshop's premise, that evaluating AI-generated claims is the bottleneck, is not an abstraction to us. It is the single reason our headline number is honest.

References

[1] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*, 2019. URL `https://arxiv.org/abs/1810.12894`.

[2] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590:580–586, 2021. doi: 10.1038/s41586-020-03157-9.

[3] Stephane Hatgis-Kessell and Emma Brunskill. When are LLMs sufficient policy optimizers for sequential RL tasks? *arXiv preprint arXiv:2605.30719*, 2026.

[4] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. ShinkaEvolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025.

[5] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[6] Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018. doi: 10.1613/jair.5699.

[7] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[8] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

[9] Haoran Tang, Rein Houthooft, Davis Foote, Adam Stooke, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. #exploration: A study of count-based exploration for deep reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL `https://arxiv.org/abs/1611.04717`.

[10] Chengjia Wang, Lanling Tang, Ming Yuan, Jiongchi Yu, Xiaofei Xie, and Jiajun Bu. Leveraging llm agents for automated video game testing. *arXiv preprint arXiv:2509.22170*, 2025.